ZERO — Operations & Architecture Guide

How the system works, what runs where, and how to diagnose failures

Contents

1. Big picture 1
2. What runs where 2
3. Runbook — starting everything 3
4. Troubleshooting — from the log line to the fix 5
5. Always-on setup (systemd) — the permanent fix 6
6. Configuration reference (`config.yaml` on the Pi) 6
7. Known issues and tuning 7
8. Repo map 7

How the whole system works, what runs where, how the two machines talk, exact commands to start, stop, and restart everything, and how to diagnose failures from the log lines alone. Read this before touching anything.

The two machines referenced throughout this doc:

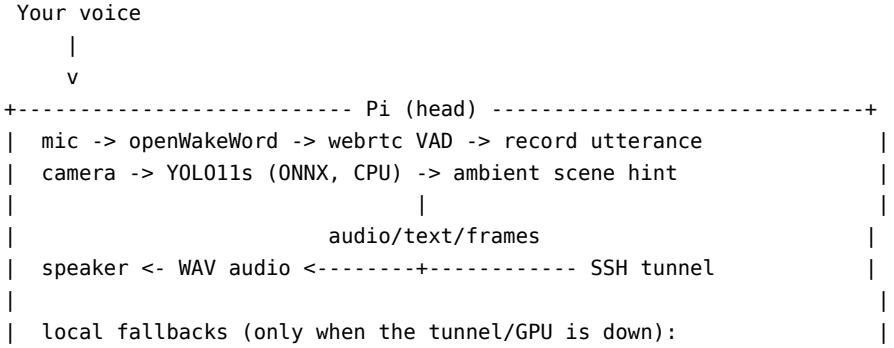
Role	Host	Shell prompt looks like	Repo path
Pi / frontend — the device you talk to	`head`	`head@head:~/Mzee/offline_v5$`	`/Mzee/offline_v5`
GPU node — where all the models run	`zerolabs1`	`obilasam3@zerolabs1 in …/zero`	`~/zero`

Rule #1 of this system: check your prompt before pasting a command. Nearly every outage so far came from running a Pi command on the GPU box, or the reverse — killing a healthy server, or tunneling a machine to itself.

---

1. Big picture

ZERO is a conversational voice robot. The Pi is the body — mic, speaker, camera, wake word, VAD — and the GPU node is the brain — STT, LLM, TTS, depth. The two are connected by one SSH tunnel carrying four port-forwards. Nothing is exposed to the internet; all traffic rides the encrypted SSH link.



```
|    STT: whisper.cpp base.en (CPU, ~27s)   TTS: Piper (amy voice)    |
+--------------------------|-------------------------------------+
                           |  autossh  (scripts/pi_tunnel.sh)
                           |  11435->11434  9000->9000  9100->9100  8000->8000
+-------------------------- GPU (zerolabs1) -------------------------+
|  :11434  Ollama            -> gemma4:latest (9.6 GB, multimodal)     |
|  :9000   Whisper server    -> large-v3-turbo (faster-whisper, CUDA)  |
|  :9100   Orpheus TTS       -> quantized GGUF via llama.cpp + SNAC    |
|  :8000   Vision server     -> Depth Anything V2 (+ optional VLM)     |
|  RTX 5060 Ti 16 GB · CUDA 13.x · venv at ~/zero/.venv (py3.12)       |
+----------------------------------------------------------------------+
```

The conversation loop (Pi, `zero/main.py`)

```
IDLE --(wake word "hey jarvis")--> LISTENING --(end of speech)--> THINKING --> SPEAKING --> LISTENING ...
```

- After one wake word the conversation flows — no wake word needed between turns. It sleeps after 30 s of silence (`conversation.sleep_timeout_ms`) or a stop phrase such as "goodbye" or "go to sleep".
- Replies stream sentence by sentence: the LLM streams tokens, the orchestrator splits sentences, and each sentence is synthesized and played while the next is still generating. A pre-synthesized filler ("Hmm, let me think.") sometimes covers the first second.
- Barge-in: while ZERO speaks, the mic stays live listening for the wake word — say it to cut ZERO off. Only sentences actually heard are kept in history.
- Memory: durable facts and one-line episode summaries are extracted by the LLM at conversation end, in a background thread, into `zero_memory.sqlite`, and injected once at the start of the next conversation.
- Vision: the camera and YOLO loop run continuously from startup. Ordinary turns get a cheap text hint ("in view: a person, a cup"). Visual questions ("what am I holding?") also attach two recent keyframes so the multimodal LLM can actually look. Live preview: `http://127.0.0.1:8008/` on the Pi.

Design principle: graceful degradation

Every remote stage has a local fallback, built lazily on first failure and retried continuously so quality returns the moment the tunnel does:

Stage	Primary (GPU)	Fallback (Pi, when tunnel/GPU down)	How you notice
STT	large-v3-turbo, ~1 s	whisper.cpp `base.en` on CPU, ~27 s	painfully slow transcription
LLM	`gemma4:latest` via Ollama	none — turns simply fail	`Ollama request failed` errors
TTS	Orpheus (natural `tara` voice)	Piper (`amy` voice)	the voice changes — Piper is the degraded-mode signal
Vision depth	Depth Anything on GPU	local detections only, no distances	silent

If you hear the Piper voice, the GPU path is broken. Piper speaking is a symptom, not the system working as intended.

---

2. What runs where

GPU node (`zerolabs1`, `~/zero`)

Port	Service	File	Model
11434	Ollama	system install (`/usr/local/bin/ollama`)	`gemma4:latest` (9.6 GB, multimodal), `moondream` also pulled
9000	Whisper STT	`server/whisper_server.py`	`deepdml/faster-whisper-large-v3-turbo` CUDA fp16
9100	Orpheus TTS	`server/orpheus_cpp_server.py`	quantized Orpheus GGUF (llama.cpp) + SNAC vocoder (onnxruntime)
8000	Vision	`server/vision/app.py` (uvicorn)	Depth Anything V2 (+ optional Qwen2-VL)

All bind to `127.0.0.1` except vision, which binds `0.0.0.0` — reachable only through the tunnel. Logs live at `~/zero_logs/{whisper,orpheus,vision,ollama}.log`.

Pi (`head`, `~/Mzee/offline_v5`)

- `python -m zero.main` — the whole robot: wake, VAD, camera, playback, conversation loop.
- The SSH tunnel (`scripts/pi_tunnel.sh`).
- Local fallback engines: Piper binary + `en_US-amy-medium` voice, whisper.cpp `base.en`, YOLO11s ONNX (always local — detection is on the Pi by design).
- `:8008` — camera preview MJPEG stream, local only.

Connectivity: the tunnel

`scripts/pi_tunnel.sh`, run on the Pi, opens one SSH connection with four local forwards. The app only ever talks to `127.0.0.1` — the tunnel makes the GPU look local:

Pi (localhost)	→	GPU (localhost)	Service
11435	→	11434	Ollama — note the port shift: 11435 locally so a Pi-local Ollama on 11434 wouldn't clash
9000	→	9000	Whisper
9100	→	9100	Orpheus
8000	→	8000	Vision

These local ports are exactly what `config.yaml` points at (`llm.host`, `stt.remote_url`, `tts.orpheus.url`, `vision.gpu.url`). It uses `autossh` (auto-reconnect across Wi-Fi blips) when installed, and plain `ssh -N` otherwise. The script runs in the foreground — leave its terminal open.

Prerequisite, one-time: the Pi must SSH to the GPU without a password.

```
# on the Pi — must print "ok" before anything else can work
ssh obilasam3@zerolabs1 'echo ok'
# if "Permission denied (publickey)":  ssh-copy-id obilasam3@zerolabs1
# if "could not resolve hostname":     use the GPU's IP, or add an /etc/hosts entry
```

---

3. Runbook — starting everything

Order matters: GPU servers first, then the tunnel, then the app.

Step 1 — GPU node (`zerolabs1`)

```
cd ~/zero
bash scripts/run_gpu_servers.sh
```

That one script starts everything the GPU needs — whisper on `:9000`, orpheus on `:9100`, vision on `:8000`, and Ollama on `:11434` if it isn't already running. It's idempotent: anything already listening on its port is skipped, so it's always safe to re-run. Each server launches detached (`setsid nohup`) and survives logout; logs go to `~/zero_logs/`.

The 2-second false alarm: the script prints a port check about 2 seconds after launch, but whisper and orpheus take 10–20 s to load their models onto the GPU. Seeing `:9000 NOT up yet` immediately after start is normal. The real check:

```
sleep 15
ss -ltnp | grep -E ':(11434|9000|9100|8000)'   # want four LISTEN lines
tail -n 5 ~/zero_logs/whisper.log               # must END on "Uvicorn running", not "Shutting down"
```

Manual equivalents, what the script runs, if you ever need just one:

```
python server/whisper_server.py --model large-v3-turbo --port 9000
python server/orpheus_cpp_server.py --port 9100
cd server/vision && uvicorn app:app --host 0.0.0.0 --port 8000
ollama serve                    # usually already up; model: ollama pull gemma4:latest
```

Direct smoke tests on the GPU, bypassing the tunnel and app, which prove the servers alone are working:

```
curl -s http://127.0.0.1:9100/health                       # {"ok":true}
curl -s -X POST http://127.0.0.1:9100/tts \
  -H 'Content-Type: application/json' \
  -d '{"text":"Hello, I am Zero.","voice":"tara"}' -o /tmp/t.wav
ls -l /tmp/t.wav                                           # ~200 KB = real audio
curl -s http://127.0.0.1:11434/api/tags | head -c 300      # lists gemma4
```

Step 2 — Pi (head), terminal 1: the tunnel

```
cd ~/Mzee/offline_v5
GPU_HOST=obilasam3@zerolabs1 bash scripts/pi_tunnel.sh
```

Leave it running. Verify from a second terminal:

```
ss -ltnp | grep -E ':(11435|9000|9100|8000)'   # four LISTEN lines held by ssh/autossh
curl -s http://127.0.0.1:9100/health           # {"ok":true} = tunnel + orpheus both good
```

Occasional `channel N: open failed: connect failed: Connection refused` lines in the tunnel terminal are not the tunnel failing — they mean a forwarded connection reached the GPU and that specific server wasn't listening yet, often because it's still loading. Don't Ctrl-C the tunnel over these.

Step 3 — Pi, terminal 2: the app

```
cd ~/Mzee/offline_v5
source .venv/bin/activate
python -m zero.main           # or:  python -m zero.main --text  (brain-only, no audio)
```

What a healthy startup log looks like

```
stt.remote     remote STT -> http://127.0.0.1:9000/transcribe
tts.remote     remote TTS (orpheus) -> http://127.0.0.1:9100/tts (voice=tara)
main           pre-synthesized 8 fillers across 3 categories      <- via Orpheus, no fallback line
llm.ollama     warming up gemma4:latest (loading into RAM)...
llm.ollama     LLM warm and pinned in RAM                          <- THE key success line
vision.eyes    eyes open — perceiving continuously
main           ZERO ready. Say the wake word to start talking.
```

Per-turn, healthy: `stt.remote heard: '...'` (about 1 s), LLM first token in about 1 s, first audio out in 1–2 s, Orpheus `tara` voice.

Red flags — each means the GPU path for that stage is broken:

- `tts.fallback primary TTS down` / `Piper ready`
- `stt.fallback primary STT failed`, followed by a 20–30 s `Inference time`
- `LLM warmup failed` / `Ollama request failed`

Restarting and stopping

```
# GPU — restart one crashed server: kill it, re-run the idempotent script
pkill -f orpheus_cpp_server.py            # or whisper_server.py / 'uvicorn app:app'
bash scripts/run_gpu_servers.sh

# Pi — restart the tunnel
pkill -f autossh; pkill -f 'ssh -N'
GPU_HOST=obilasam3@zerolabs1 bash scripts/pi_tunnel.sh

# Pi — restart the app: Ctrl-C, then  python -m zero.main
```

Never run `pkill -f orpheus/whisper/uvicorn` on the GPU box as "cleanup" — that kills the production servers. This exact mistake caused a full outage: the graceful `Shutting down` line in `whisper.log` was our own stray `pkill`.

---

4. Troubleshooting — from the log line to the fix

Symptom (exact log text)	Meaning	Fix
`Connection refused` on 9000/9100/11435 (Pi)	Nothing listening on the Pi's local port — tunnel not running, or that server is down on the GPU	Start the tunnel (Step 2); check the GPU server
`RemoteDisconnected` / `Connection reset by peer` (Pi)	Tunnel is up, the connection reached the GPU, but the server crashed mid-request	Check that server's log on the GPU; restart it
`bind ... Address already in use` when starting the tunnel	An old tunnel still holds the ports	`pkill -f autossh; pkill -f 'ssh -N'`, then restart
`Permission denied (publickey)` from `pi_tunnel.sh`	No SSH key, or the script was run on the GPU box (SSH to itself)	Check your prompt; `ssh-copy-id` from the Pi
`channel N: open failed: connect failed` in tunnel terminal	Harmless: a forward reached the GPU, that server wasn't up yet	Leave the tunnel alone; fix or wait for the server
`:9000 NOT up yet` right after `run_gpu_servers.sh`	The 2-second check fired before the model finished loading	Wait about 15 s, re-check with `ss`
`RuntimeError: llama_decode returned -1` in orpheus.log	Orpheus llama.cpp decode failure, seen after the process has served many requests (stale KV state). Not VRAM — the 16 GB card runs about 5.5 GB total	Restart orpheus; if it recurs, see Known Issues
`whisper.log` ends with `Shutting down`, no error	Graceful SIGTERM — someone `pkill`'d it	Restart; find who killed it, usually a wrong-box cleanup
`stt.fallback ... building local fallback` + ~27 s transcribe	Remote STT unreachable, so the Pi fell back to CPU whisper	Fix the tunnel or the whisper server
Piper voice instead of `tara`	Remote TTS unreachable or crashing	Fix the tunnel or orpheus; it retries every sentence and recovers on its own
`Ollama request failed ... 11435`	Tunnel down, or Ollama down on the GPU	Check the tunnel; then `ss -ltnp | grep 11434` on the GPU
`Could not open camera index 0` (Pi)	Another process holds `/dev/video0`, usually a not-fully-dead earlier `zero.main`	`fuser -v /dev/video0`, kill the stale PID. Voice still works without the camera
`GetGpuDevices ... /sys/class/drm` warnings (Pi)	onnxruntime probing for a GPU the Pi doesn't have	Ignore — cosmetic; CPU provider is intended on the Pi
`use gpu = 1 ... no GPU found` in whisper.cpp output (Pi)	The local fallback whisper noting the Pi has no GPU	Ignore — only appears when the fallback is active at all

End-to-end diagnostic order

Work from the GPU outward; each step isolates one layer.

```
# 1. GPU: are all four servers listening?           (on zerolabs1)
ss -ltnp | grep -E ':(11434|9000|9100|8000)'
# 2. GPU: does each answer directly?                (on zerolabs1)
curl -s http://127.0.0.1:9100/health ; curl -s http://127.0.0.1:9000/health
# 3. Pi: is the tunnel holding the local ports?     (on head)
ss -ltnp | grep -E ':(11435|9000|9100|8000)'
# 4. Pi: does a request make it through the tunnel?  (on head)
curl -s http://127.0.0.1:9100/health
# 5. Only now start or restart the app.
```

---

5. Always-on setup (systemd) — the permanent fix

Manual terminals and hand-run `pkill` are how every outage here has happened. The units in `scripts/systemd/` make both boxes self-starting and self-healing. `Restart=always` means a crashed server or dropped tunnel comes back on its own, including a fresh, state-free orpheus after a `llama_decode` crash.

Edit `User=`, `WorkingDirectory=`, `ExecStart=` paths, and `GPU_HOST=` in each unit first — they ship with `CHANGEME`. Real values: GPU user `obilasam3`, repo `/home/obilasam3/zero`; Pi user `head`, repo `/home/head/Mzee/offline_v5`.

```
# GPU node (zerolabs1) — ollama already has its own ollama.service if installed via the installer
sudo cp scripts/systemd/zero-{whisper,orpheus,vision}.service /etc/systemd/system/
sudo systemctl daemon-reload
sudo systemctl enable --now zero-whisper zero-orpheus zero-vision

# Pi (head) — tunnel first, app ordered after it
sudo cp scripts/systemd/zero-tunnel.service scripts/systemd/zero.service /etc/systemd/system/
sudo systemctl daemon-reload
sudo systemctl enable --now zero-tunnel zero
journalctl -u zero -f
```

After this, “restart everything” becomes:

```
sudo systemctl restart zero-whisper zero-orpheus zero-vision   # GPU
sudo systemctl restart zero-tunnel zero                        # Pi
```

---

6. Configuration reference (`config.yaml` on the Pi)

One file selects every engine. The factory (`zero/factory.py`) maps names to classes, so swapping an engine never touches code. A gitignored `config.local.yaml` deep-merges on top for per-device overrides.

Key	Current value	Meaning
`wake.model` / `threshold`	`hey_jarvis` / 0.5	wake word — a custom “hey zero” would mean training an openWakeWord model
`vad.*`	webrtc, aggressiveness 3, 700 ms silence	endpointing; `min_utterance_rms` gates out far-away voices
`stt.engine`	`remote` → `http://127.0.0.1:9000/transcribe`	GPU whisper through the tunnel
`stt.fallback`	`whispercpp` (base.en)	local CPU failover, built lazily
`llm.host`	`http://127.0.0.1:11435`	tunnel port for GPU Ollama (11434 remotely)
`llm.model`	`gemma4:latest`	8B multimodal model, pulled on the GPU
`tts.engine`	`orpheus` → `http://127.0.0.1:9100/tts`, voice `tara`	GPU TTS (voices: tara, leah, jess, leo, dan, mia, zac, zoe)

Key	Current value	Meaning
`tts.fallback`	`piper`	local failover voice, and the audible degraded-mode signal
`vision.enabled` / `multimodal`	`true` / `true`	camera loop on; keyframes go to the LLM on visual turns
`vision.gpu.enabled`	`false`	depth facts off — the LLM sees frames directly instead
`memory.enabled`	`true` → `zero_memory.sqlite`	facts and episode summaries persisted across sessions
`voiceid.enabled`	`false`	owner-only voice gate — enroll first with `python test_voiceid.py enroll`

Persona and system prompt live in `zero/llm/persona.py` (`SYSTEM_TEMPLATE`).

---

7. Known issues and tuning

1. **Orpheus's SNAC vocoder runs on CPU**, about 5 s per sentence. Its log shows `CUDAExecutionProvider is not in available provider names` — the GPU venv has CPU `onnxruntime`, not `onnxruntime-gpu`. The llama.cpp half is on GPU; only vocoding is CPU. Fix on the GPU, after verifying a CUDA-13-compatible build exists for the card: `pip uninstall -y onnxruntime && pip install onnxruntime-gpu`

2. **Orpheus can die with `llama_decode returned -1`** after serving many requests across app restarts — stale KV state, confirmed not VRAM, since the 16 GB card sits at about 5.5 GB with everything loaded. A restart clears it, and systemd's `Restart=always` makes that automatic. If it ever crashes on the first sentence after a fresh start, suspect llama.cpp CUDA kernels on the RTX 5060 Ti (Blackwell / sm_120) instead — rebuild `llama-cpp-python` against the current CUDA, or run orpheus with `--n-gpu-layers 0` (CPU, slow but stable).

3. **`gemma4:latest` is a locally-created Ollama tag.** It exists in `ollama list` on the GPU; it isn't a public registry name. If it's ever missing after a re-install, re-create it or point `llm.model` at another pulled model.

4. **Camera contention on the Pi.** A previous `zero.main` that didn't fully die can hold `/dev/video0`, and the next run comes up voice-only. Run `fuser -v /dev/video0` and kill the stale PID.

5. **First Orpheus and whisper start downloads models** from Hugging Face — GGUF plus SNAC, and CT2 whisper. The GPU node needs internet access for that first run only.

6. **Latency knobs:** filler probability (`conversation.filler_probability`), `llm.max_tokens` at 500 to cap reply length, and history trimming (`history_turns` / `history_trim_at`) to keep the prompt cache warm.

---

8. Repo map

```
zero/                 the robot (runs on the Pi)
  main.py             state machine + conversation loop (streaming, barge-in, fillers)
  factory.py          config -> engine objects (THE swap point)
  config.py           config.yaml + config.local.yaml loader
  wake/ vad/ stt/ llm/ tts/ vision/ voiceid/ memory/ audio/
                      one package per stage, each behind a small base interface;
                      stt/ and tts/ contain remote_engine (GPU client) + fallback wrappers
  llm/persona.py      system prompt (SYSTEM_TEMPLATE)
server/               the brain (runs on the GPU node)
  whisper_server.py   POST /transcribe  (faster-whisper large-v3-turbo, CUDA)
  orpheus_cpp_server.py  POST /tts, /tts_stream, GET /health  (GGUF llama.cpp + SNAC)
```

```
  vision/app.py       depth + scene facts (+ optional VLM /analyze)
scripts/
  run_gpu_servers.sh  start everything on the GPU (idempotent, detached, logs ~/zero_logs)
  pi_tunnel.sh        the SSH tunnel, Pi -> GPU (autossh; foreground)
  setup_pi.sh         one-pass fresh-Pi install (local stack only)
  systemd/            always-on units for both boxes (+ README)
config.yaml           every knob (see section 6)
docs/                 GPU_OFFLOAD_PLAN.md (design rationale) · VISION.md · PROGRESS.md
zero_memory.sqlite    long-term memory (Pi, gitignored)
```